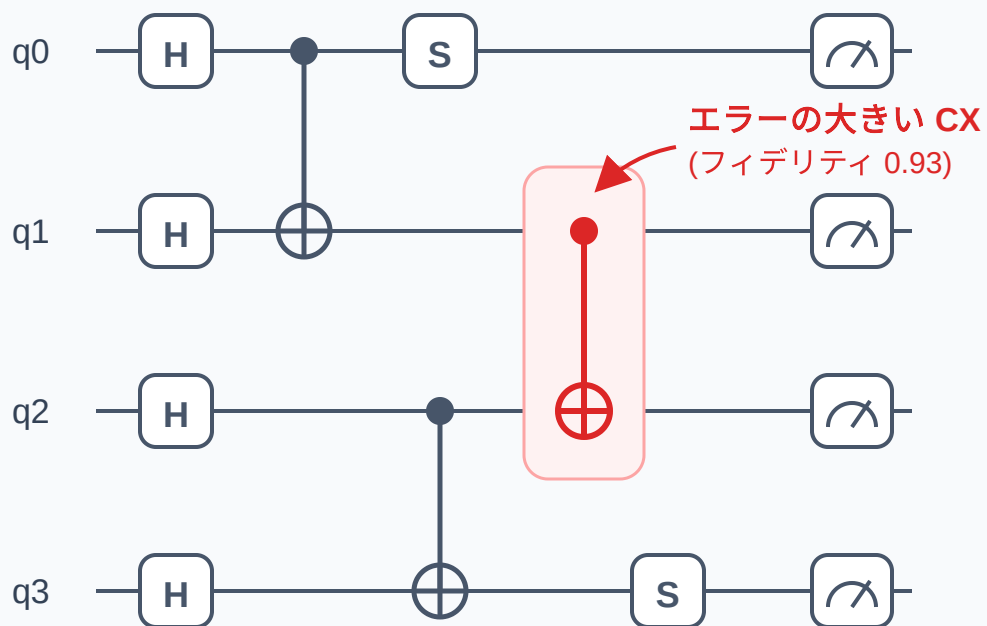


課題:大きな量子回路は、いまの量子コンピュータでそのまま動かすと精度が出ない



横線 = 量子ビット / 箱 = 1量子ビットゲート / ●—⊕ = 2量子ビットゲート(CX)
右端の計器 = 測定 / ゲートは左から右の順に適用される

1 2量子ビットゲート(CX)はエラーが大きい

1量子ビットゲートの成功率(フィデリティ)が 99.9% 程度なのに対し、CX は 9 割台(箇所によってはさらに低い)。回路全体の精度をほぼ CX が決める。

2 チップ上の場所によって品質にムラがある

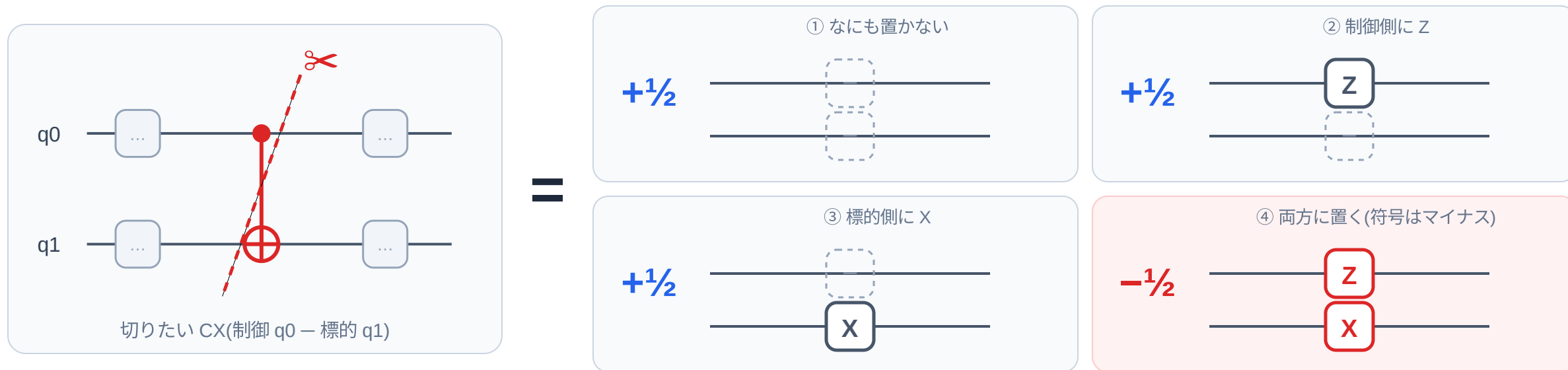
同じ機械でも「この量子ビットペアの CX だけ特に悪い」ことがある。品質は定期的な較正(キャリブレーション)で実測されている。

3 この研究の視点:回路切断を「エラー抑制」に使う

回路切断はもともと「大きな回路を小さなマシンに載せる」ための技術。ここではそれを「品質の悪い CX を回避して精度を上げる」目的に転用する。

発想 – ゲートカッティング: 品質の悪い CX ゲートを回路から「切除」し、CX を含まない複数の小さな回路を実行。その結果を古典コンピュータで合成して、**元の回路と同じ答え(期待値)を復元**する。

原理: CX ゲートは「1量子ビットゲートだけの回路」 4つの足し算に置き換えられる



どの置き換え後の回路にも 2量子ビットゲートがない → q0 側と q1 側は完全に独立。4つの回路を別々に実行して測定し、結果(期待値)を係数付きで足し合わせると元の回路の答えになる:

$$\langle \text{元の回路} \rangle = \frac{1}{2} \langle ① \rangle + \frac{1}{2} \langle ② \rangle + \frac{1}{2} \langle ③ \rangle - \frac{1}{2} \langle ④ \rangle$$

代償: 実行する回路の数が増える。

1箇所切ると4通り、k箇所切ると 4^k 通り(2箇所では16、3箇所では64)。切る場所は厳選が必要 → 次ページ。類似手法の Wire Cutting(配線を切る)は 16^k がかかるため、 4^k で済む Gate Cutting の方が実用的。

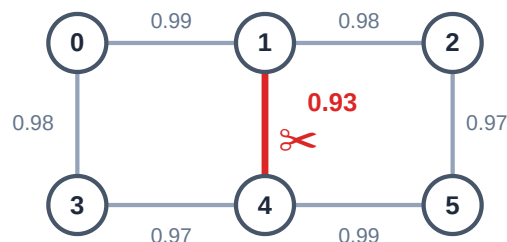
どこを切るか:実機の校正データ + 数理最適化(MILP)で自動選択

入力①

実機の校正データ `device.json`

量子ビットごと・CX ペアごとの実測フィデリティ(成功率)。

```
{
  "qubits": [{id: 0, fidelity: 0.999}, ...],
  "couplings": [
    {control: 0, target: 1, fidelity: 0.99},
    {control: 1, target: 4, fidelity: 0.93},
    ...
  ]
}
```



○ = 量子ビット / 線 = CX とそのフィデリティ

最適化

整数計画ソルバー(SciPy MILP)で切断対象を決める

「回路中のどの CX 命令を切るか」を 0/1 変数にして、次のルールで最適化:

- ✓ フィデリティがしきい値(例 0.96)を下回る CX ほど「切ると得」と評価する
- ✓ 切れるのは最大 `max_cuts` 本まで(4^k の実行コスト爆発を抑える)
- ✓ 「2 グループ分けをまたぐ CX は必ず切る」という形で回路の分割も表現できる(完全な分離は強制しない)

ポイント:回路は「量子ビット = 点、CX = 線」のグラフとして扱う。同じ量子ビットペアに CX が複数あっても、**命令番号で 1 個ずつ** 区別して狙い撃ちできる。

入力②

実行したい量子回路

Qiskit で書いた回路(実験ではランダム Clifford 回路を使用)。

出力

切断対象リスト `CutTarget`

「回路の何番目の命令の CX(どの量子ビットペア)を切るか」の具体的なリスト。

```
CutTarget(instruction_index=12,
          qubits=(1, 4), fidelity=0.93)
```

これを次ページの「展開・実行」エンジンに渡す。

全体パイプライン:このリポジトリ(src/gate_cutting/)の構成

入力 ① 量子回路 – Qiskit 回路(実験用にはランダム Clifford 回路を生成)
circuits.py

入力 ② device.json – 実機の校正データを読み込み、フィデリティをノイズ確率(`ErrorParams`)へ変換 device.py

STEP 1

切る CX を選ぶ

校正データをもとに SciPy MILP で切断対象(`CutTarget`)を自動選択。

mip.py / cut_selection.py



STEP 2

4^k 個のサブ回路に展開

切る CX を「なし / Z / X / Z&X」の 4 パターン(係数 $\pm\frac{1}{2}$)に置き換えた回路を生成。

gate_cutting.py



STEP 3

ノイズ付きシミュレーション

Stim 形式に変換し、各ゲートの後ろに実機由来のノイズを挿入して高速にサンプリング実行。「切っても結果を復元できるか」の事前評価にも使う。

stim_backend.py



STEP 4

古典後処理で答えを復元

各サブ回路の測定から期待値(全ビットのパリティ)を計算し、係数を掛けて合算 → 元の回路の期待値。

gate_cutting.py: run_gate_cut()

検証方法:同じ回路を切らずに実行(`run_standard`)した期待値と、ゲートカッティング経由(`run_gate_cut`)の期待値を比較し、切る数を増やすほど理想値との誤差が減る(= エラー抑制が効く)ことを確認する。

ゴール:論文用の実験スクリプトを再利用可能なコンポーネントに整理し、量子計算プラットフォーム **OQTOPUS** に組み込めるようにする(ワイヤーカッティング対応などは今後)。

補足:シミュレータに Stim を使う理由

Stim はクリフォード回路専用の高速シミュレータ。状態ベクトルを持たず「スタビライザー形式」で計算するため、量子ビット数が増えても多項式時間で動く。この実験の性質と 3 点で噛み合う。

1 4^k × ショットの物量に耐える

k 箇所切ると 4^k 個のサブ回路 × 各 10,000 ショット (+ 切らない基準実行との比較)。状態ベクトル方式はコストが量子ビット数 n に対して 2^n で増えるが、Stim は多項式時間で、ショットもまとめて高速にサンプリングできる。

2 ノイズモデルがそのまま載る

挿入するノイズはパウリノイズのみ(ゲート後の DEPOLARIZE、測定前の読み出し反転)。パウリノイズはクリフォードの世界を壊さない。CX 分解の置き換えゲートも Z/X(パウリ)なので、**切った後のサブ回路もすべて Stim で扱える形のまま**。

3 「切って大丈夫か」の事前評価

切り方によっては期待値が消えて復元に失敗する(奇数分割問題)。実行前にシミュレータで検査し、ダメなら切らない、という安全弁を入れている。8×8 = 64 量子ビットの実機規模でこの事前検査ができるのはスタビライザー方式だからこそ。

代償:クリフォードゲート(X/Y/Z/H/S/Sdg/CX など)しか扱えない。実験はクリフォード回路に限定し、回転ゲートは S 等に置き換える。目的は「エラー抑制が効くか」のノイズ検証なのでこの構成で十分。ノイズなし・小規模での手法比較には Qiskit Aer を併用している。